

Project Title: File Integrity Monitoring Tool

Group # 10

Members: Harris Waheed, Lovepreet Sidhu

Course: INFO 3171 S10

Date: March 30, 2025

Project Overview:

Objective:

To design and implement a File Integrity Monitoring (FIM) system using Java, applying core cybersecurity principles such as integrity and availability through SHA-256 hashing and real-time file event detection. The goal is to demonstrate how unauthorized file changes such as malware, insider threats, and accidental edits can be detected and logged using a lightweight, custom monitoring tool.

Additionally, the project includes a research component, where we:

- Explore OSSEC, a widely used open-source Host-based Intrusion Detection System (HIDS), as a real-world FIM solution.
- Compare features such as alerting, scalability, and configurability.
- Evaluate pros and cons of using a custom lightweight tool vs. deploying a mature enterprise-grade solution like OSSEC.

Problem Statement:

Unauthorized file changes can compromise system integrity. This project implements a Java-based File Integrity Monitoring (FIM) tool and evaluates OSSEC as a potential industry solution.

Background Information:

File Integrity Monitoring (FIM) tools are used extensively in system security, specifically in an area called SIEM or security information and event management (IBM, 2025). FIM tools aid in identifying malicious actions, unintended alterations, and internal threats affecting system files. FIM's are commonly employed to protect important files of databases and system configuration files of operating systems in enterprise environments. FIM's are part of the Integrity portion of the

CIA (Confidentiality, Integrity, Availability) triad as they ensure files have not been corrupted, modified or deleted based upon a baseline hash of the file stored and analyzed by the FIM. Additionally, to verify file changes, a FIM compares its cryptographic hash files to see if any changes have taken place. It does this by performing a cryptographic hash function, which takes an argument of n-length and returns a fixed-length value in its place (Rosulek, 2017). There are many hash functions that FIM's can utilize, however currently, SHA-256 is the industry standard for security; originally created by the United States government.

Many FIM's already exist as part of a more comprehensive Intrusion Detection System (IDS), however there are a few standalone solutions such as Qualys FIM, a cloud-based FIM and OSSEC, a free and open-source host-based IDS with a built-in FIM.

The practical applications of using a FIM are many. Firstly, forensic analysis of files, where professionals may use a FIM to gather data about files. Additionally, preventing polymorphic and metamorphic viruses as a FIM creates a hash of a file and if there is a virus that changes its code, the FIM can alert the network administrator that a file's hash has changed. FIM tools can also be used in legal settings to see if part of the law database or archive has been altered. This is done by creating hashes of every file, which can be time consuming if automation is not used (Imtiaz, 2024).

Implementations:

We developed a custom **File Integrity Monitoring (FIM)** system in Java that utilizes **SHA-256 cryptographic hashing** to report file content changes. The program continuously monitors a specified directory in real-time using Java's built-in **WatchService API**, and notifies the user

whenever a file is created, modified, or deleted. For demonstration purposes, the monitored folder simulated sensitive content, such as files labeled “Secret” or a text file containing example passwords. Upon detecting an event, the system computes and displays the current SHA-256 hash of the file. While it does not persist or compare hashes against a saved baseline over time, the system provides immediate visibility into changes, offering a lightweight form of integrity awareness. As the test files were manually created and modified during development, the system was evaluated in a **simulated environment**, unlike enterprise FIM deployments where file structures and content are governed by operational policies. We also reviewed **OSSEC**, a professional open-source FIM tool, for comparison and broader context, although it was not integrated or used as our prototype.

Tools & Technologies

- Java Development Kit Se-22 and IntelliJ IDEA were used for development.
- Java WatchService API which enabled real-time directory monitoring.
- SHA-256 algorithm via Message Digest ensured secure cryptographic hash generation.
- Terminal, PowerShell was used to simulate file changes.
- OSSEC was studied as an industry-level tool and compared to
- VirtualBox Ubuntu 20 Virtual Machine was used for installing OSSEC locally.

Development Process

We are developing a tool that generates SHA-256 hashes as a baseline and monitors a folder for file changes using Java’s WatchService API. As development progresses, we are simulating file tampering to test detection and alert functionality. Error handling is being added to manage missing

or unreadable files. We are also exploring OSSEC's features for comparison with our custom solution.

Security Measures:

SHA-256 hashing is used to ensure file integrity by detecting unauthorized modifications. Real-time alerts are being implemented to notify the user of any file activity. The tool is designed to run in a restricted environment without requiring administrative access, making it safe for testing (Rosulek, 2017).

Project Team Contributions:

Both team members are equally involved in all aspects of the planning and research of the project. Lovepreet performed the FIM simulated experiments and Harris conducted research and documented results. Communication was done via WhatsApp and project management was done through Microsoft OneDrive. We are currently dividing the other work collaboratively, contributing to: Writing and testing the Java-based File Integrity Monitoring system, implementing SHA-256 hashing and real-time file change detection. Also, researching OSSEC for comparison with our custom tool.

Results and Outcomes:

We started with an empty test file in the watched directory while tests show system's monitoring active. We modified the test file; the system created a new cryptographic hash and alerted us that the file had been altered (figure 1 below). After creating a hash, the file contents were modified and saved. The expected results of the FIM are the new and old hash are different (figure 2). We

deleted that file, and observed no hash stored or operation displayed, which was expected. Subsequently, we created a new file called test2. We now expect to see the hash and the operation on the directory, which was observed (figure 3).

After testing OSSEC, an enterprise FIM and comparing the results with our own lightweight FIM, we concluded that both were similar in their capabilities and speeds, however OSSEC's user interface was easier to interact with through the terminal. File integrity, whether custom-made or integrated into an enterprise solutions are often similar, with the primary difference sometimes only being in the preferred user interface.

Figures:

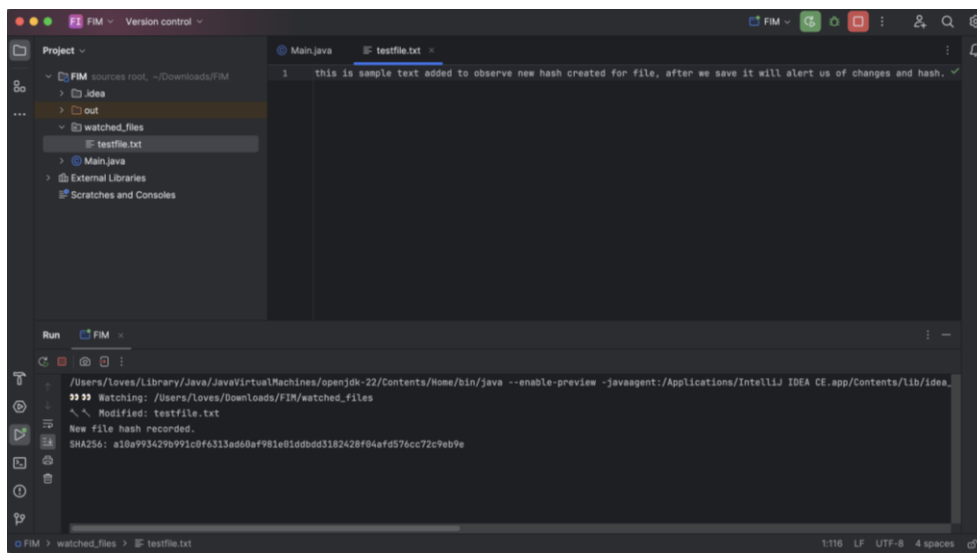


Figure 1: SHA-256 hash performed on text file by our FIM.

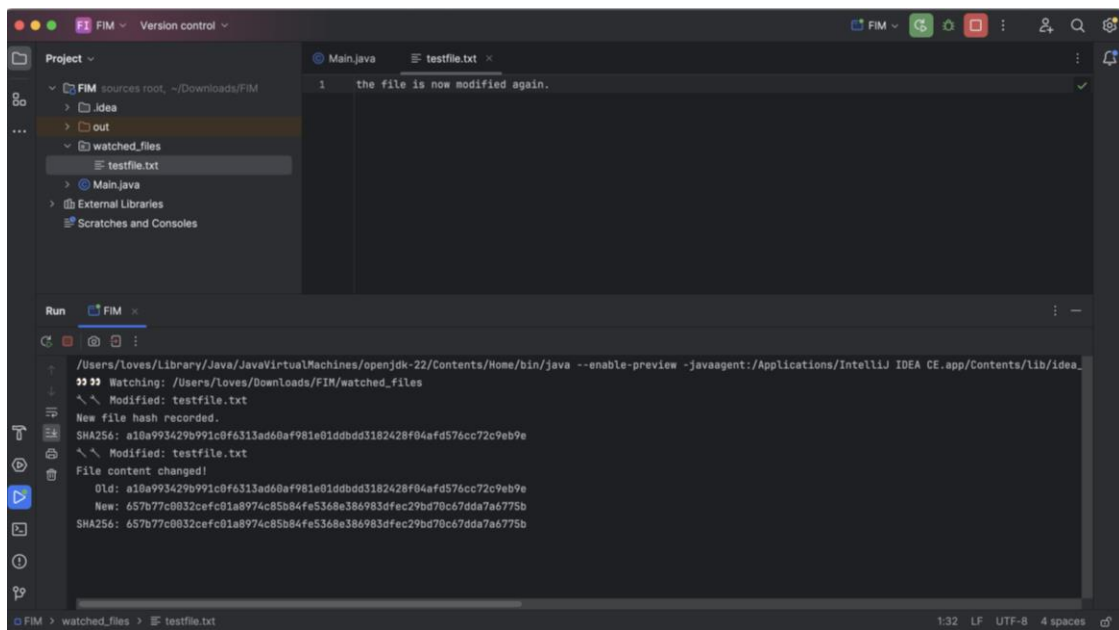


Figure 2: Modification of text file with a new SHA-256 cryptographic hash being assigned to it

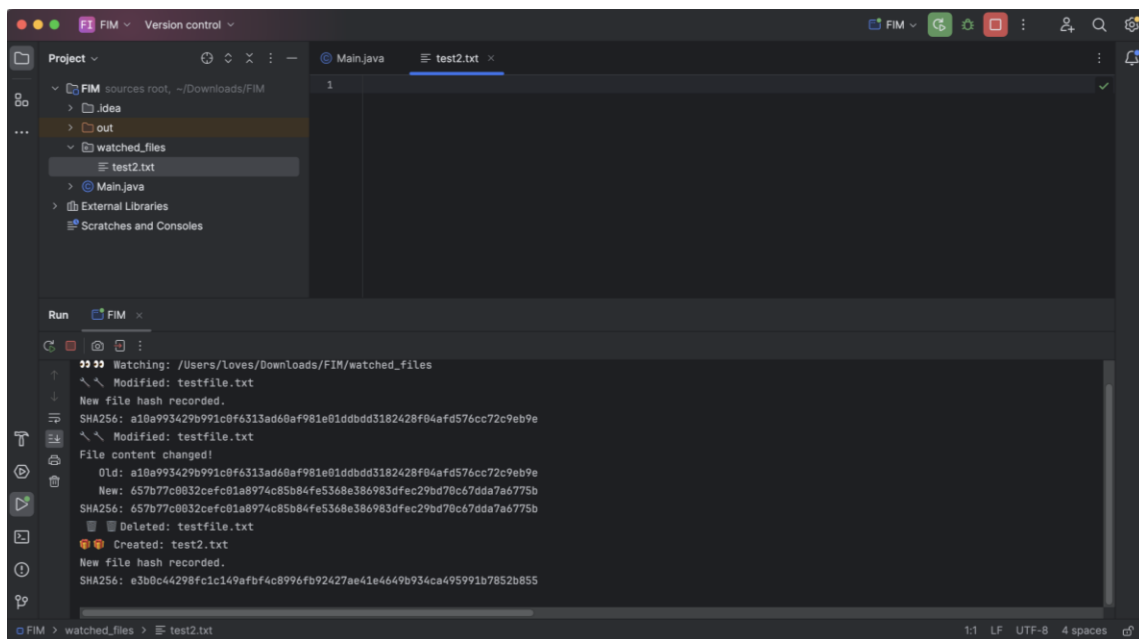


Figure 3: Deleting testfile.txt and observing a new SHA-256 hash being created for test2.txt

Conclusion and Reflections:

We leveraged Java programming; we had not previously used the WatchService API. This project gave us the opportunity to learn and apply it for the first time in a practical, real-world context, specifically for real-time file monitoring. It was a valuable learning experience that showed us how core Java features can be adapted to address cybersecurity use cases like file integrity monitoring. Additionally, we learned about verification of cryptographic hashes as a means of verifying file integrity, whereas previously we thought file integrity only meant theft or corruption of data.

A slight challenge faced was the open-source version of OSSEC was not compatible with the latest version of Ubuntu. This was solved by installing Ubuntu 20 in a virtual machine in order to run the program; the latest version is Ubuntu 24.04.

In the future, we can extend the project by automating the usage of the FIM. This can be used in an enterprise environment where we program the FIM to run every time a new file is uploaded to the server and check all file hashes stored in the server to see if there are any changes.

References:

IBM. (2025, January 13). *What is SIEM?* <https://www.ibm.com/think/topics/siem>

Oracle. (2024). *WatchService (Java SE 22 & JDK 22)*. Oracle Java Platform Standard Edition 22 Documentation. <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/nio/file/WatchService.html>

Imtiaz, K. (2024, March 4). *What is File Integrity Monitoring (FIM)?*. CrowdStrike. <https://www.crowdstrike.com/en-us/cybersecurity-101/exposure-management/file-integrity-monitoring/>

Rosulek, M. (2017). *Chapter 12: Hash functions*. The Joy of Cryptography OE 1st. <https://open.oregonstate.edu/cryptographyOEfirst/chapter/chapter-12-hash-functions/>